VeriTrace

Scan. Verify. Trust.

Technical Documentation

Blockchain-based product authentication and supply-chain traceability system
(Client project: BlockAuth)

Document type	System / technical documentation
Version	1.0
Date	October 2026
Platform	Android app, Node.js backend, Ethereum (Hardhat), Firebase, Python ML
Status	Prototype — feature complete, verified

Table of Contents

1. Introduction

2. System Overview and Features

3. System Architecture

4. Use Case Model

5. Data Flow Diagrams (Level 0, Level 1)

6. Smart Contract Design

7. Backend Design

8. Verification Engine

9. Machine Learning Anomaly Detection

10. Mobile Application Design

11. Data Model

12. Process Flows (Sequence Diagrams)

13. Security Design

14. API Reference (Summary)

15. Deployment and Operations

16. Testing and Quality Assurance

17. Limitations and Future Work

Appendix A. Demo Data · Appendix B. Glossary · Appendix C. Technology Stack

1. Introduction

1.1 Purpose

This document describes the design, components, behaviour and technical implementation of **VeriTrace**, a prototype system that lets any consumer verify whether a physical product is genuine by scanning a QR code, and lets businesses in a supply chain record every custody transfer of that product on a blockchain. It is intended for developers, technical reviewers, maintainers and the client's technical stakeholders.

1.2 Problem Statement

Counterfeit goods (electronics, pharmaceuticals, cosmetics, luxury items) are routinely sold with convincing packaging. Traditional labels and holograms can be copied, and centralised product databases can be edited by whoever controls them. Consumers have no independent way to check where a product came from or whether its record has been altered.

1.3 Solution Summary

VeriTrace anchors a cryptographic **fingerprint** (SHA-256 hash) of each product's identity data and the complete **custody history** in a smart contract. A QR code on the product carries the product ID and the fingerprint. When scanned, the backend compares the QR, the product database and the blockchain record, checks the custody history, scan patterns and an ML anomaly model, and returns one of four results: **Genuine**, **Flagged**, **Suspect** or **Unverified**, together with the reasons and the product's full journey.

1.4 Scope

- In scope: product registration, QR generation and labelling, two-step custody transfers, sale recording, public verification, role-based administration, reporting, ML anomaly scoring, demo tooling.
- Out of scope: payment, inventory quantities, iOS, production key management, physical tamper-evident tags.

1.5 Definitions

Term	Meaning
Fingerprint / product hash	SHA-256 of the canonical JSON of a product's immutable identity fields, stored on-chain as `bytes32`.
Custody / holder	The business that currently possesses the product according to the blockchain.
Handoff	One recorded custody event (registered, transfer initiated, received, cancelled, sold).
Custodial wallet	An Ethereum account derived and held by the backend on behalf of a business user.

2. System Overview and Features

2.1 Actors

Actor	Authentication	Responsibilities
Consumer	None	Scans a product or enters its ID; reads the verification result and journey.
Manufacturer	Email/password	Registers products, prints/shares QR labels, ships products to suppliers.
Supplier	Email/password	Confirms received shipments; ships onward to another supplier or a retailer.
Retailer	Email/password	Confirms received shipments; marks products as sold.
Administrator	Email/password	Creates business accounts, assigns roles (synchronised on-chain), deactivates accounts, views reports.

2.2 Functional Features

- **Product registration** with validated form (name, brand, category, batch, serial, manufacture/expiry dates, MRP, origin, description); automatic ID `VT-<YYYY>-<NNNNNN>`.
- **QR code** generation, PNG sharing and A4 label-sheet printing (3 × 7 labels).
- **Two-step custody transfer** (initiate → confirm), cancellation, multi-hop distribution, sale recording.
- **Public verification** by camera scan or manual ID, with reasons, product details, journey timeline and blockchain references.
- **Role administration** mirrored on-chain; account deactivation revokes on-chain rights.
- **Reports**: scans per result, recent alerts, products by stage, accounts by role, ML statistics.
- **ML anomaly detection** that can raise a result to Flagged (never Suspect) and degrades gracefully.

2.3 Non-functional Characteristics

Quality	How it is addressed
Integrity	Fingerprints and custody history immutable on-chain; rules enforced by contract and API.
Privacy	No personal data on-chain; anonymous device IDs; no location permission in the app.
Availability	ML is optional; verification continues if ML is down. Clear errors when chain/database are unreachable.
Usability	Consumers need no account or wallet; every status uses colour + icon + text.
Testability	Pure verification rules, automated unit, integration and end-to-end suites.

3. System Architecture

VeriTrace follows a three-tier architecture. The Android app is a thin client; all business logic, blockchain interaction and database access are centralised in the backend. The mobile app never talks to the blockchain directly.

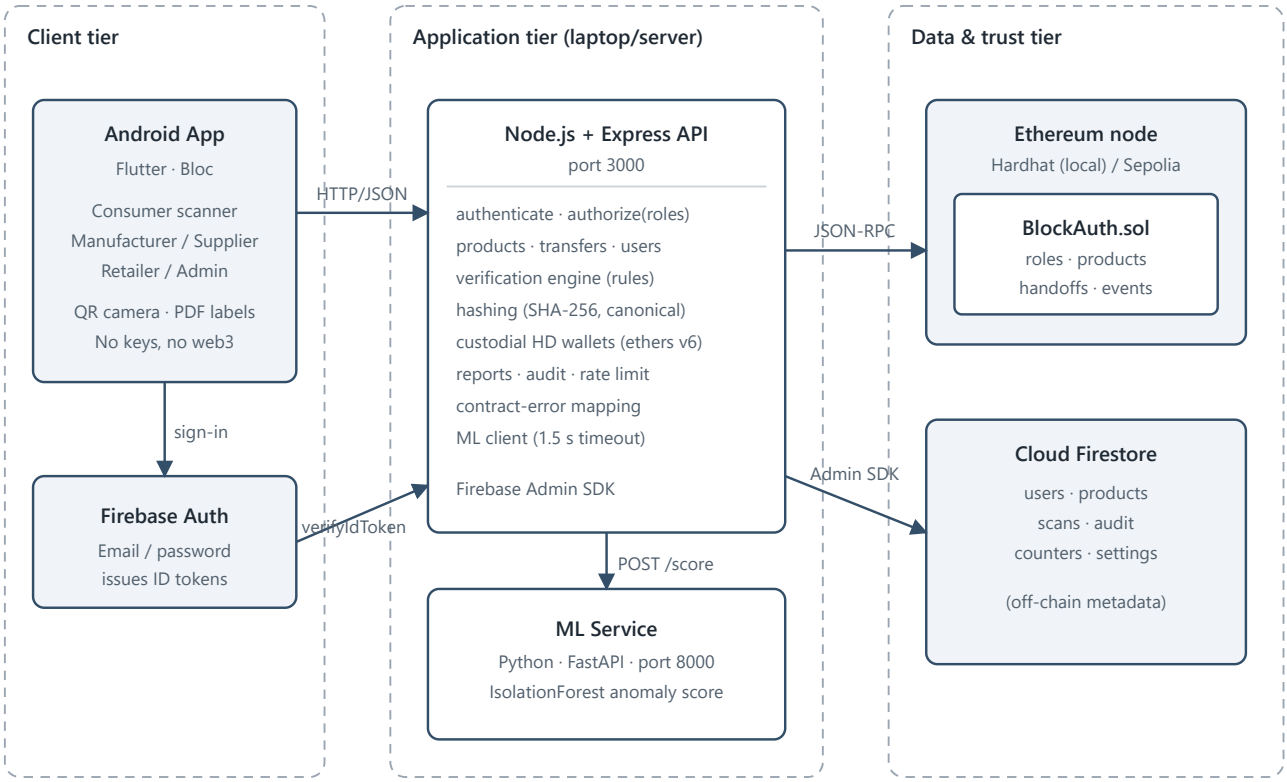


Figure 3.1 — System architecture

3.1 Component Responsibilities

Component	Technology	Responsibility
Mobile app	Flutter 3, Dart, flutter_bloc, go_router, dio, mobile_scanner	User interface for all actors; camera scanning; QR rendering, sharing and printing.
Backend API	Node.js 20+, Express, ethers v6, firebase-admin, zod	Authentication, authorization, validation, business rules, blockchain transactions, verification, reports.
Smart contract	Solidity 0.8.24, Hardhat	Authoritative roles, product fingerprints, custody state machine, handoff history, events.
Database	Cloud Firestore	User profiles, product metadata, scan log, audit trail, counters, runtime settings.
Identity	Firebase Authentication	Email/password sign-in for business users; ID tokens verified by the backend.
ML service	Python, FastAPI, scikit-learn	Anomaly score for a product's journey and scan pattern.

3.2 Design Principles

- **Chain as source of truth** for roles, custody and fingerprints; Firestore holds descriptive data and logs.
- **Defence in depth**: every write is checked by API middleware and again by the contract against `msg.sender`.
- **Thin client**: the app holds no secrets; consumers need neither an account nor a wallet.
- **Single source of truth for rules**: one verification module and one hashing function.

4. Use Case Model

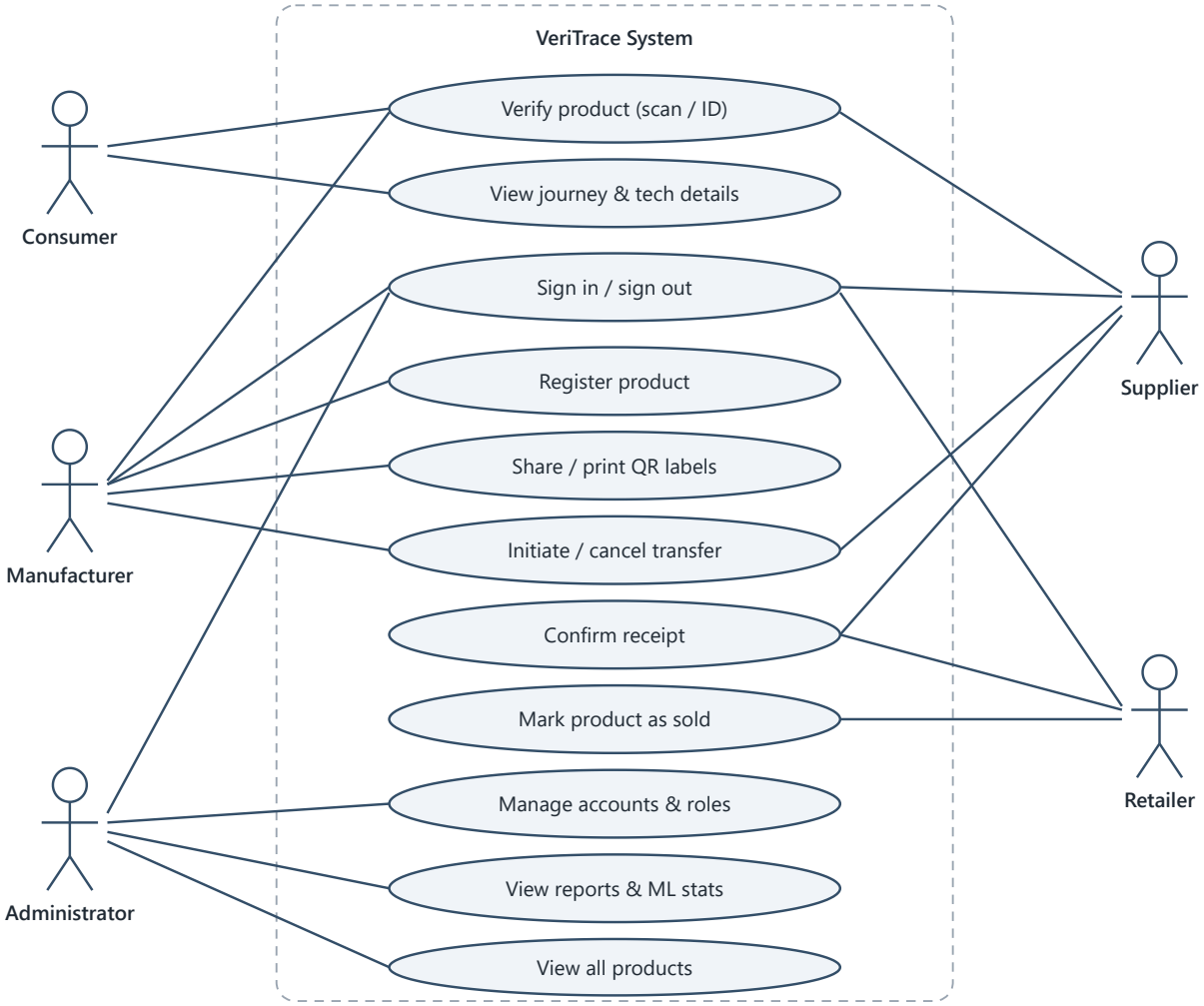


Figure 4.1 — Use case diagram (business actors may also verify products)

4.1 Key Use Case Descriptions

Use case	Actor	Main flow	Alternative / error flows
UC-1 Verify product	Consumer (any)	Scan QR → app sends payload → backend evaluates rules → result with reasons, journey, technical details.	Non-VeriTrace QR rejected in scanner; unknown ID → Unverified; server unreachable → error with retry.
UC-2 Register product	Manufacturer	Fill form → backend validates, allocates ID, hashes, calls `registerProduct`, stores metadata → QR shown.	Validation errors per field; duplicate ID → 409; chain failure → readable message.
UC-3 Initiate transfer	Manufacturer, Supplier	Choose recipient → confirm → `initiateTransfer` signed by holder's wallet → product "Transfer pending".	Invalid transition (e.g. Manufacturer→Retailer) rejected by contract and shown as "Rejected by the smart contract".
UC-4 Confirm receipt	Supplier, Retailer	Incoming list → Confirm → `confirmReceipt` → custody moves.	Only the named recipient may confirm; cancelled transfers cannot be confirmed.
UC-5 Mark as sold	Retailer	In stock → Mark as sold → confirm → `markSold`; terminal state.	Not holder / not at retailer / already sold → rejected.
UC-6 Manage accounts	Administrator	Create account (Auth user + wallet + on-chain role + funding); change role; deactivate.	Email exists → 409; admin cannot modify own role.

5. Data Flow Diagrams

5.1 Context Diagram (DFD Level 0)

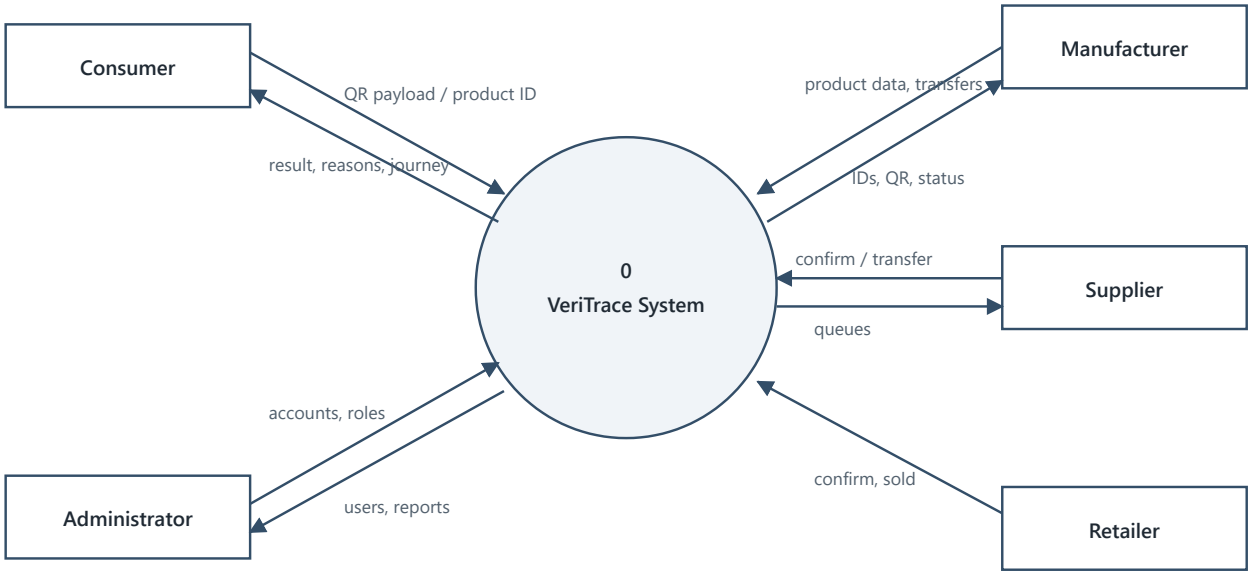


Figure 5.1 — Context diagram

5.2 DFD Level 1

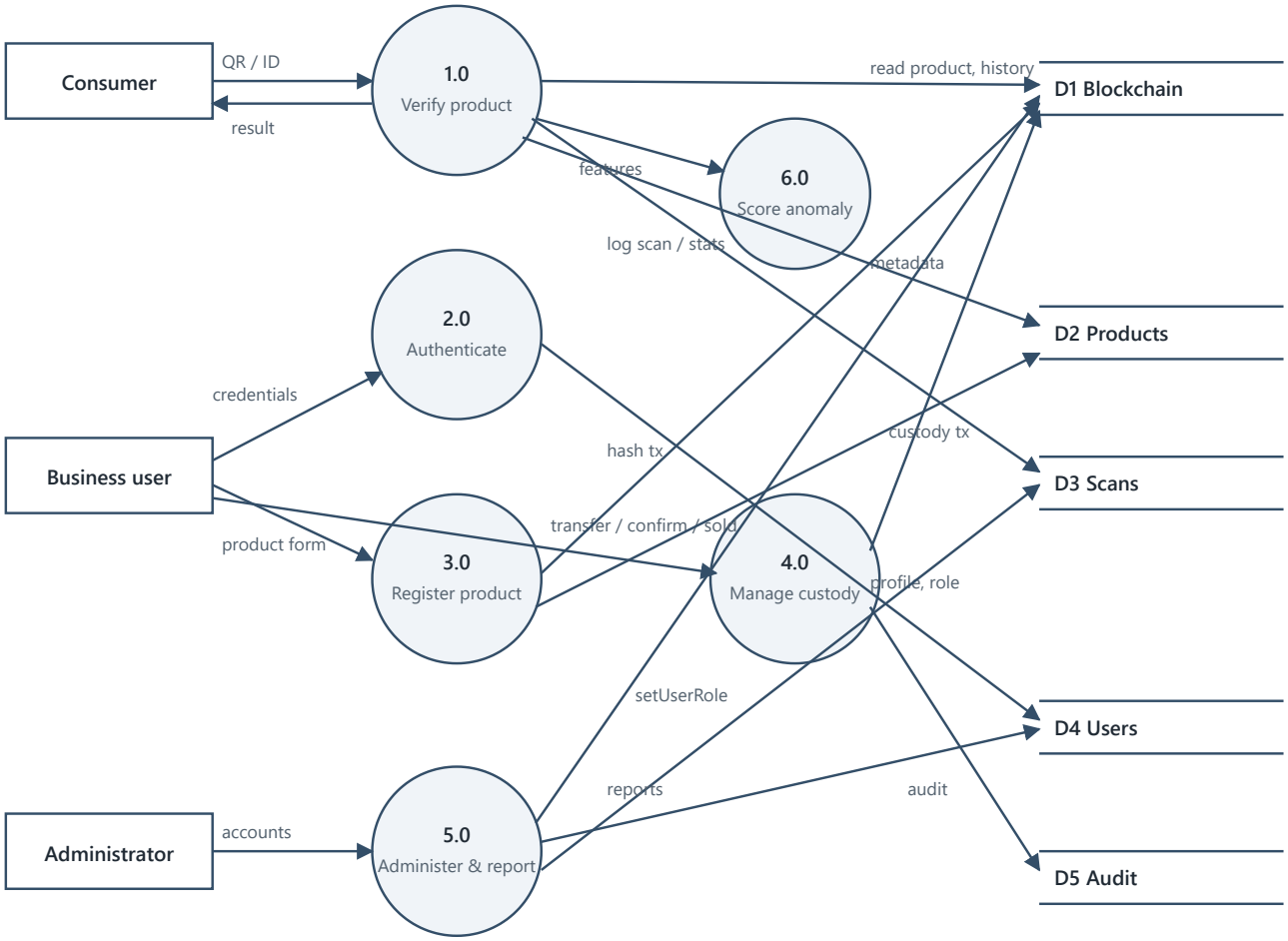


Figure 5.2 — Level 1 data flow diagram

Process	Inputs	Outputs / stores touched
1.0 Verify product	QR payload or ID, device ID	Reads D1, D2, D3; writes D3; calls 6.0; returns status, reasons, journey.
2.0 Authenticate	Email, password (Firebase), ID token	Reads D4 for profile and role.
3.0 Register product	Product form	Writes D1 (fingerprint), D2 (metadata), D5.
4.0 Manage custody	Transfer, confirm, cancel, sold actions	Writes D1 (state + handoff), D5.
5.0 Administer & report	Account data, role changes	Writes D1 (roles), D4, D5; reads D2, D3 for reports.
6.0 Score anomaly	Journey/scan features	Returns anomaly score and flag.

6. Smart Contract Design

The contract `BlockAuth.sol` (Solidity 0.8.24) is the trust anchor. It stores only non-personal data: wallet roles, product keys, fingerprints, custody state and history.

6.1 Data Structures

```
enum Role      { None, Admin, Manufacturer, Supplier, Retailer }
enum Stage     { None, Manufactured, AtSupplier, AtRetailer, Sold }
enum EventType { Registered, TransferInitiated, Received, TransferCancelled, Sold }

struct Product { string productId; bytes32 productHash; address manufacturer;
                 uint64 registeredAt; address currentHolder; address pendingRecipient;
                 Stage stage; bool exists; }
struct Handoff { address from; address to; uint64 timestamp; EventType eventType; Stage stageAfter;
}

mapping(address => Role)        roles;
mapping(bytes32 => Product)     products;    // key = keccak256(productId)
mapping(bytes32 => Handoff[])   histories;
```

6.2 Functions

Function	Caller	Effect
`setUserRole(user, role)`	Admin	Assigns role; cannot change own role; emits RoleChanged.
`registerProduct(id, hash)`	Manufacturer	Creates product at Manufactured; rejects duplicates and zero hash.
`initiateTransfer(id, to)`	Current holder	Sets pending recipient if the transition is valid.
`confirmReceipt(id)`	Pending recipient	Moves custody; stage becomes AtSupplier or AtRetailer.
`cancelTransfer(id)`	Current holder	Clears pending transfer.
`markSold(id)`	Holding retailer	Stage Sold (terminal).
`getProduct`, `getHandoffHistory`	Anyone (view)	Read state; unknown IDs return `exists=false`.

6.3 Product State Machine

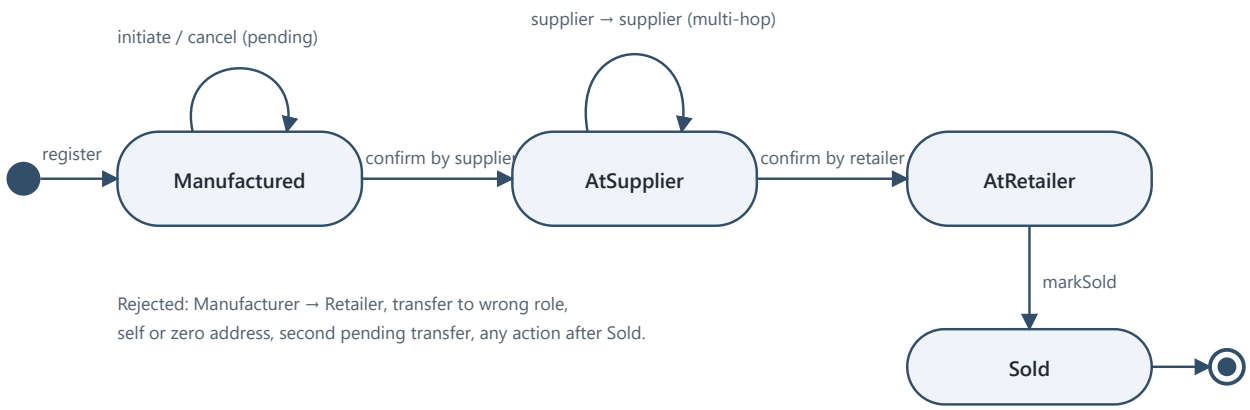


Figure 6.1 — Custody state machine enforced on-chain

6.4 Events and Errors

Each state change emits exactly one event (`ProductRegistered`, `TransferInitiated`, `TransferReceived`, `TransferCancelled`, `ProductSold`, plus `RoleChanged`). The backend matches the n-th event to the n-th handoff to attach transaction hashes and block numbers. Reverts use custom errors (e.g. `InvalidTransition`, `NotCurrentHolder`, `NotPendingRecipient`, `ProductAlreadyExists`) which the backend maps to readable messages.

7. Backend Design

7.1 Module Structure

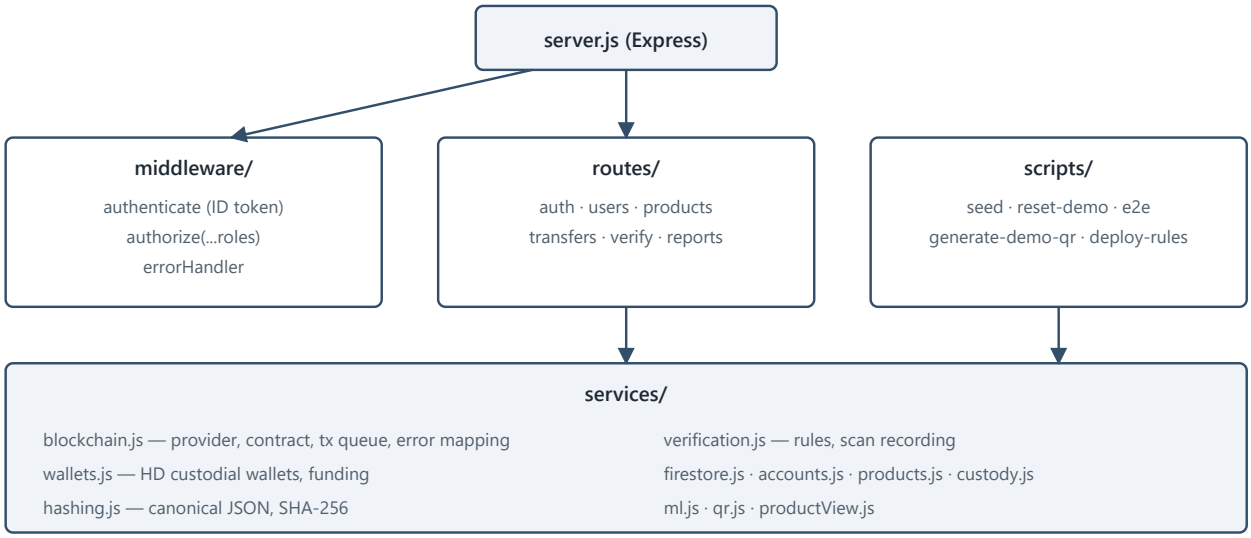


Figure 7.1 — Backend module structure

7.2 Request Pipeline

1. CORS and JSON body parsing; request logging.
2. **authenticate**: verifies the Firebase ID token and loads the Firestore profile; rejects missing, expired, profile-less or deactivated accounts.
3. **authorize(roles)**: rejects callers whose role is not permitted (HTTP 403).
4. Input validation with **zod** schemas (field-level messages).
5. Service logic; blockchain writes are signed with the **acting user's own wallet**.
6. Uniform error format `{ error: { code, message, details } }`; contract reverts become `CHAIN_REJECTED` with the decoded reason.

7.3 Custodial Wallets

Each business account receives an Ethereum wallet derived from a backend-held mnemonic at path `m/44'/60'/0'/0/<index>`. Index 0 is the deployer/admin, indices 1–9 are reserved for demo accounts, and new accounts start at index 10. Wallets with a low balance are automatically funded by the deployer. Transactions are serialized per wallet to prevent nonce collisions.

7.4 Product Identity and Fingerprint

Product IDs are allocated in a Firestore transaction on `counters/products`. The fingerprint is computed in one function: the identity fields *productId, name, brand, category, batchNumber, serialNumber, manufactureDate, expiryDate, countryOfOrigin, manufacturerId* are normalised (trimmed strings, empty → null, dates as YYYY-MM-DD), serialised as JSON with alphabetically sorted keys and no whitespace, and hashed with SHA-256. The QR payload is `veritrace://verify?id=<productId>&h=<hash>`.

8. Verification Engine

All decision logic is in `services/verification.js`. Each rule produces a reason with a severity; the final status is the most severe level present. This makes the outcome independent of rule order and guarantees that ML (always "warning") can never produce Suspect.

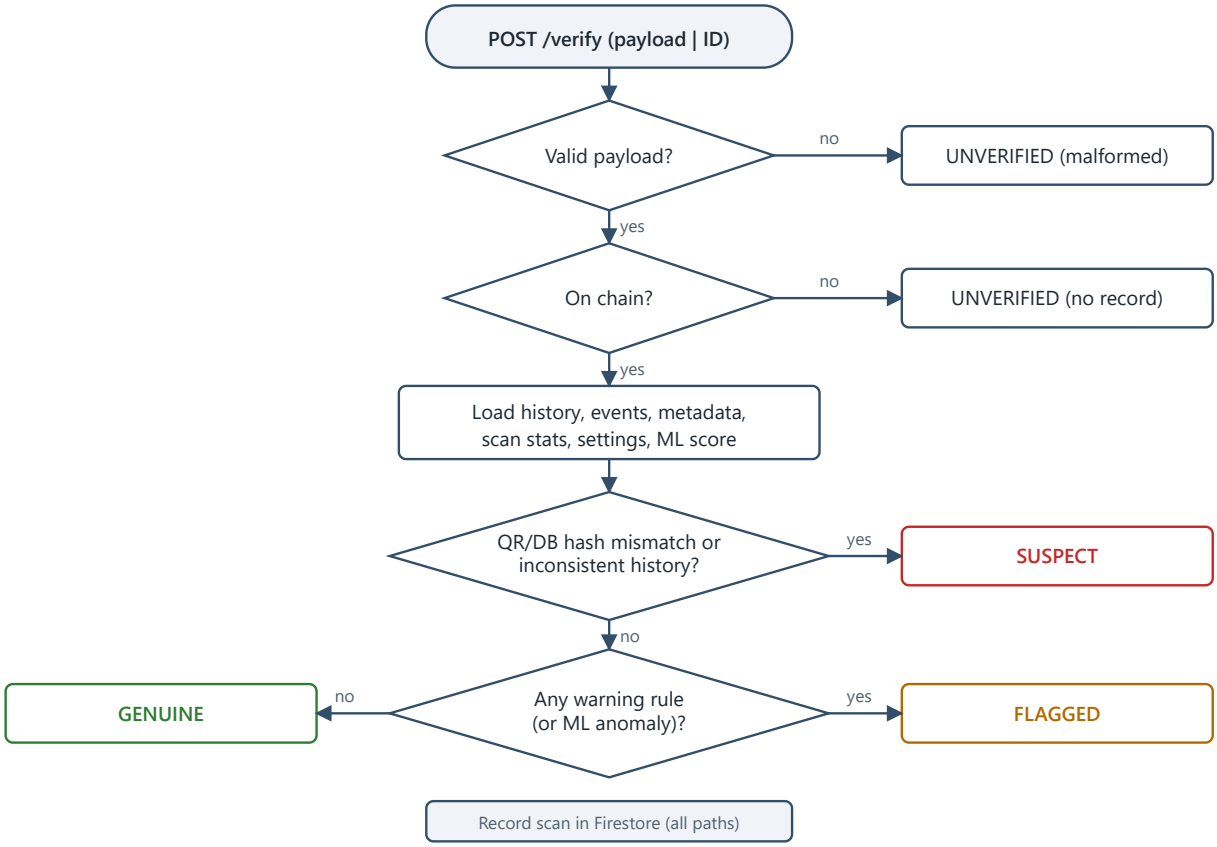


Figure 8.1 — Verification decision flow

8.1 Rules

Code	Severity → result	Condition
MALFORMED_PAYLOAD	Unverified	Not a VeriTrace QR / invalid ID.
NOT_ON_CHAIN	Unverified	ID unknown to the contract (never labelled "counterfeit").
QR_HASH_MISMATCH	critical → Suspect	QR fingerprint ≠ on-chain fingerprint (forged label).
METADATA_TAMPERED	critical → Suspect	Fingerprint recomputed from database ≠ on-chain (record altered).
HISTORY_INCONSISTENT	critical → Suspect	Replaying the handoffs does not reproduce the current state, or events ≠ history.
NOT_AT_RETAIL	warning → Flagged	Stage Manufactured or AtSupplier at time of scan.
TRANSFER_PENDING	warning → Flagged	A transfer is not yet confirmed.
EXPIRED	warning → Flagged	Expiry date has passed.
EXCESSIVE_SCANS / MANY_DEVICES	warning → Flagged	More than 10 scans / more than 5 distinct devices (possible clone).
FAST_HANDOFF	warning → Flagged	Custody gap below the configured minimum.
METADATA_MISSING	warning → Flagged	On chain but no database record.
ML_ANOMALY	warning → Flagged	ML service reports an anomaly.
ON_CHAIN, QR_HASH_MATCH, METADATA_MATCH, HISTORY_CONSISTENT, ML_NORMAL, AT_RETAILER, SOLD	ok	Passed checks shown to the user.

Thresholds have environment defaults and can be overridden at runtime through the Firestore document `settings/verification`. Every verification, including unverified ones, is recorded in the `scans` collection.

9. Machine Learning Anomaly Detection

9.1 Purpose

Deterministic rules catch known patterns. The ML service adds a learned, multi-feature view of what a "normal" journey and scan pattern look like, to surface unusual combinations (cloned codes, scripted scan bursts, laundering through many intermediaries, scans of unreleased goods, very old products suddenly scanned).

9.2 Model and Features

Feature	Description
numHandoffs	Number of custody receipts.
minGapSeconds, meanGapSeconds	Minimum and mean time between custody changes (log-scaled).
secondsSinceRegistration	Product age (log-scaled).
scanCount, distinctDevices, scansLastHour	Scan pattern including the current scan (log-scaled).
stage	Custody stage at scan (ordinal).

The model is a scikit-learn **IsolationForest** (300 trees) trained on 16,000 synthetic normal journeys plus ~2% injected anomalies. The decision threshold is the 99th percentile of normal training scores. On held-out synthetic data it catches about 92% of injected anomalies with about 1% false alarms.

9.3 Integration

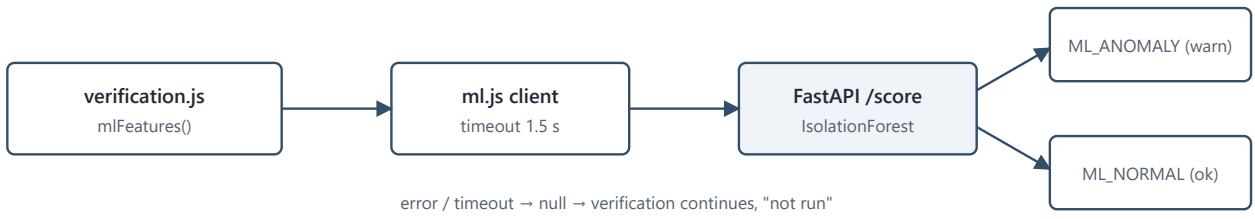


Figure 9.1 — ML integration and graceful degradation

ML results are stored per scan (`mlScored` , `ml`), shown in the app's technical details and summarised for administrators in Reports (service status, model version, scans scored, scans flagged).

10. Mobile Application Design

10.1 Architecture

The Android app is built with Flutter using a feature-first structure and the Bloc pattern for state management. Repositories wrap the HTTP API; Blocs/Cubits hold screen state; views render state.

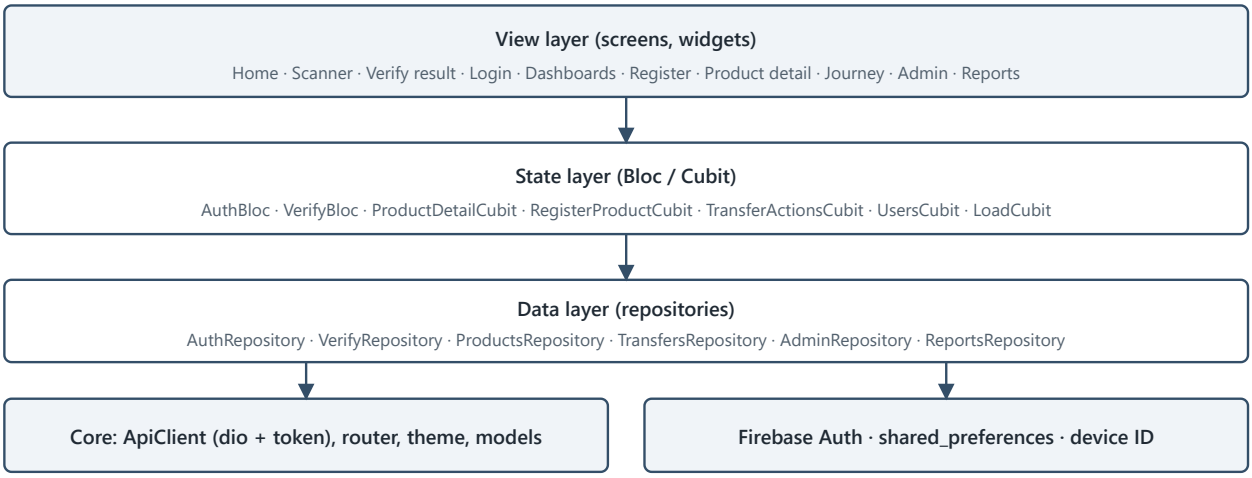


Figure 10.1 — Mobile application layers

10.2 Navigation and Access

Routing uses go_router with a guard: public routes (home, scanner, verify, login) are open; dashboards require login and the matching role; the backend enforces roles again. After sign-in the user is routed to the dashboard of their role.

10.3 Screens

Screen	Description
Home	Scan a product, Enter product ID, Business login / Open dashboard.
Scanner	Camera QR scanner with torch; rejects non-VeriTrace codes; camera-permission guidance.
Verification result	Status badge (colour + icon + text), reasons, product details, journey timeline, technical details.
Manufacturer dashboard	Product list with live stage chips; register form; label sheet printing.
Product detail	QR, Share PNG, Print labels, transfer/cancel/confirm/sold actions by role and state, journey, technical details.
Supplier / Retailer dashboards	Incoming (confirm) and In custody / In stock (transfer or mark sold).
Admin dashboard	Users (filter, create, change role, activate/deactivate, on-chain role), all products, reports with ML card.

10.4 Platform Configuration

- Package `com.veritrace.app`, minimum Android 6.0 (API 23); release APK contains arm64 and x86_64 code.
- Permissions: Internet and Camera only. Location permissions are explicitly removed from the merged manifest.
- Backend URL from build configuration, overridable at runtime from the login screen; USB mode uses `adb reverse`.

11. Data Model

11.1 Entity-Relationship Overview

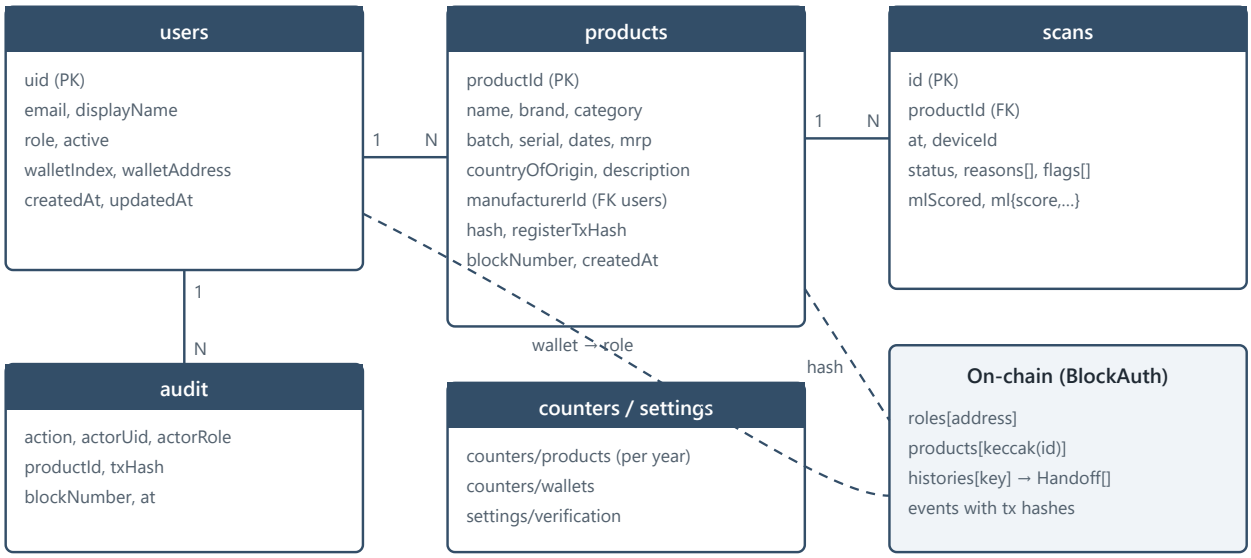


Figure 11.1 — Data model (Firestore collections and on-chain state)

Personal data (emails, names) exists only in Firebase/Firestore. The blockchain holds wallet addresses, product keys, fingerprints and custody events. Security rules deny all direct client access to Firestore except a signed-in user reading their own profile; all other access goes through the backend using the Admin SDK.

12. Process Flows

12.1 Product Registration

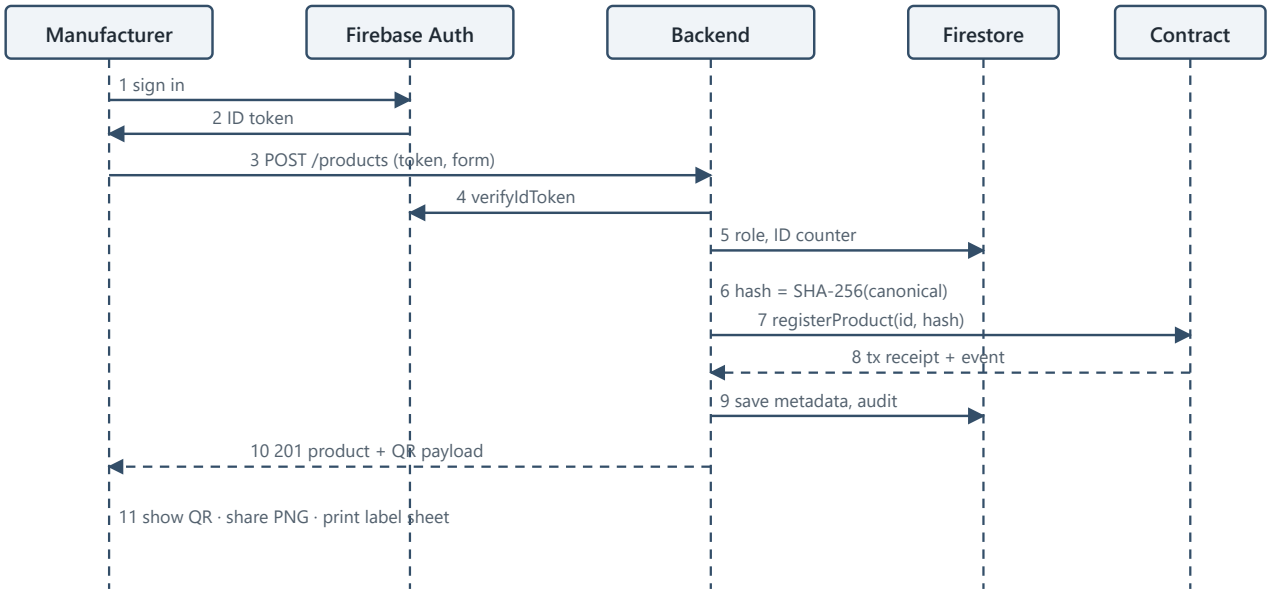


Figure 12.1 — Registration sequence

12.2 Custody Transfer

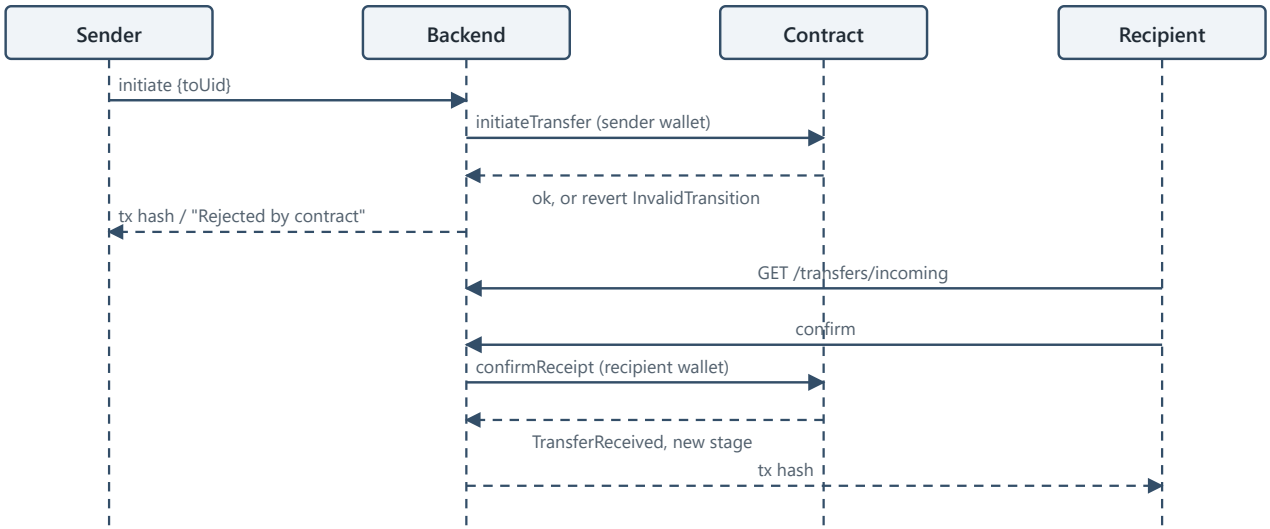


Figure 12.2 — Two-step custody transfer

12.3 Consumer Verification

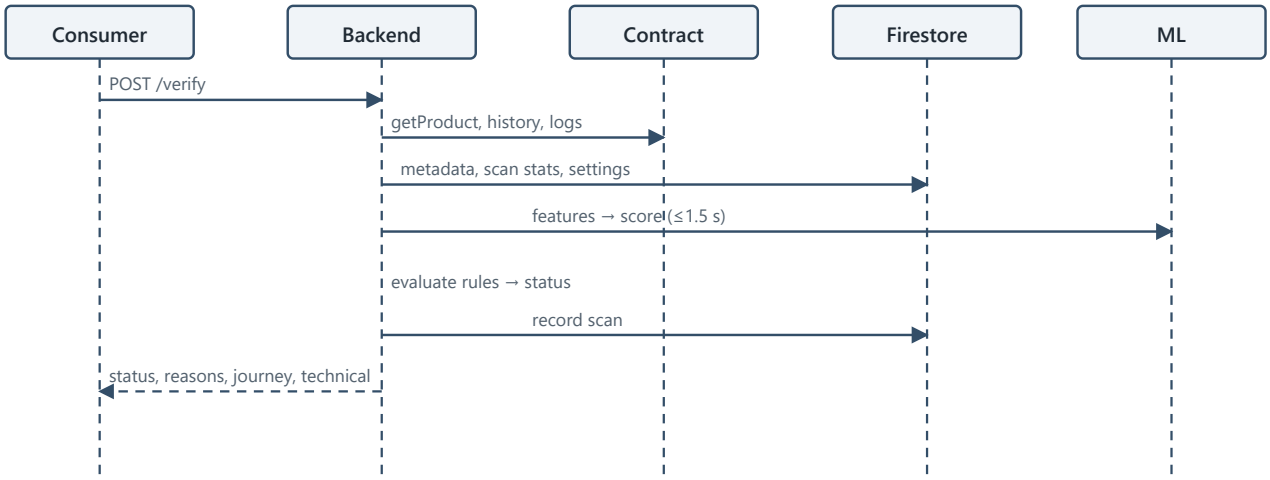


Figure 12.3 — Verification sequence

13. Security Design

Threat	Control
Forged / altered QR label	QR fingerprint compared with on-chain fingerprint → Suspect.
Database record tampering	Fingerprint recomputed from database compared with on-chain → Suspect.
Copied genuine QR (clones)	Scan-count and distinct-device thresholds, ML anomaly, pre-retail rule → Flagged.
Unauthorised writes	Firebase token + role middleware; contract checks role, holder and recipient of the signing wallet.
Illegal custody paths	State machine in the contract; reverts with typed errors.
Deactivated insiders	Profile flagged inactive, Firebase account disabled, on-chain role set to None.
Direct database access	Firestore rules deny clients; only the backend (Admin SDK) reads/writes.
Abuse of public endpoint	Per-IP rate limit on `/verify`; strict input validation.
Privacy	No personal data on-chain; anonymous device IDs; no location permission.

> **Prototype trade-offs:** custodial keys derived from one backend mnemonic, open CORS, plain HTTP on the local network and a local single-node blockchain. These are acceptable for demonstration and must be replaced for production (see Section 17).

14. API Reference (Summary)

Method & path	Access	Purpose
GET /health	Public	Chain, contract, Firestore and ML status.
POST /verify	Public (rate-limited)	Verify a QR payload or product ID.
GET /me	Business	Current profile and on-chain role.
POST /users · GET /users · PATCH /users/:uid	Admin	Create, list, update accounts (role/active synced on-chain).
GET /users/recipients?role=	Business	Recipients for transfer picker.
POST /products · GET /products/mine	Manufacturer	Register; list own products.
GET /products	Admin	All products.
GET /products/:id · /products/:id/qr-payload	Business	Detail with journey; QR payload.
POST /products/:id/sold	Retailer	Mark sold.
GET /transfers/incoming · /transfers/custody	Business	Queues derived from contract events.
POST /transfers/:id/initiate · confirm · cancel	Per role	Custody actions.
GET /reports/summary · /reports/scans	Admin	Statistics, alerts, ML summary, scan lists.

All errors use `{ "error": { "code", "message", "details" } }`. Contract rejections are returned as `CHAIN_REJECTED` with `details.reason` set to the custom error name. The full reference is in `docs/API.md`.

15. Deployment and Operations

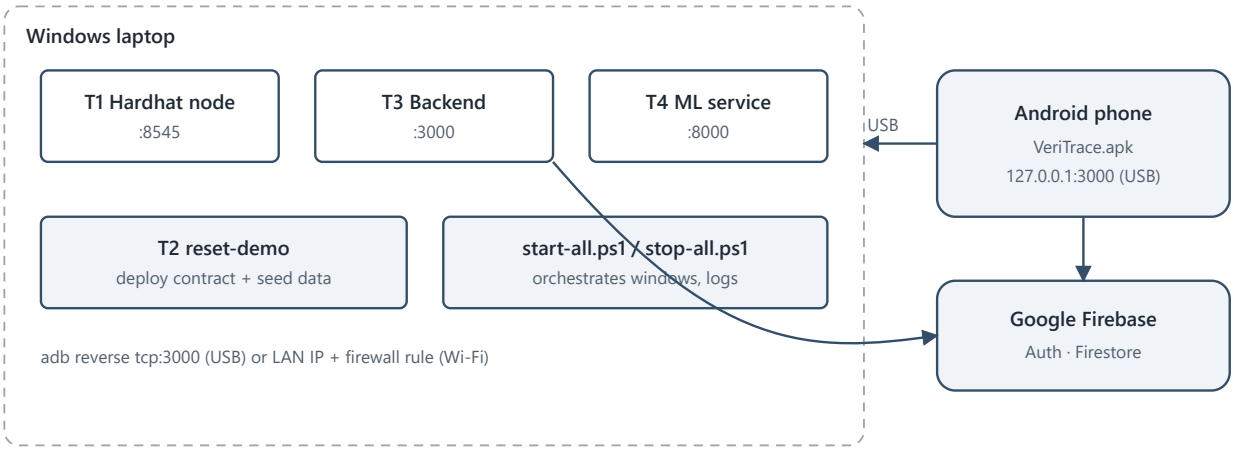


Figure 15.1 — Local deployment topology

15.1 Operating Procedure

1. One-time: install Node.js (and optionally Python); run `setup.ps1` (dependencies, ML environment, adb, configuration).
2. Each run: connect the phone by USB and run `start-all.ps1`. It starts the blockchain, backend and ML windows, redeploys the contract, seeds demo data, installs and opens the app.
3. Stop with `stop-all.ps1`. The local chain is ephemeral; each start begins from fresh demo data.

15.2 Configuration

Variable	Purpose
CHAIN, RPC_URL, MNEMONIC	Local (Hardhat) or Sepolia network and custodial mnemonic.
FIREBASE_PROJECT_ID, FIREBASE_SERVICE_ACCOUNT	Firebase project and service-account key file.
SCAN_FLAG_THRESHOLD, DEVICE_FLAG_THRESHOLD, MIN_HANDOFF_GAP_SECONDS	Verification thresholds (overridable in Firestore).
ML_URL, ML_TIMEOUT_MS	ML service address and timeout.

An optional Sepolia testnet deployment is supported via `npm run deploy:sepolia` and `CHAIN=sepolia`.

16. Testing and Quality Assurance

Suite	Tooling	Scope	Result
Smart contract	Hardhat, chai	Roles, registration, duplicates, full path, invalid transitions, cancel, after-sold, history, events	25 passing
Backend unit/integration	jest, supertest	Hashing, QR parsing, every verification rule, error mapping, auth/role gates, validation, ML client	83 passing
End-to-end	Node script on live stack	Sign-in, register, transfers, sold, verification cases, admin, reports, ML	46 / 46
Mobile	flutter_test, bloc_test, mocktail	Auth and verify blocs, routing guard, parsing, widgets	26 passing, 0 analyzer issues
ML service	pytest	Health, normal vs anomalous journeys, validation	7 passing

In addition, the release APK was installed and exercised on Android, and the client package was verified by unpacking it into a clean folder and running setup, start-up and the end-to-end suite.

17. Limitations and Future Work

17.1 Limitations

- A local single-node blockchain demonstrates the mechanism but is not independent evidence; production requires a public or consortium network.
- QR codes can be copied; cloning is detected heuristically, not prevented.
- A valid record proves the code was registered, not that the physical item is the registered one.
- Custodial keys held by one backend are a prototype shortcut.
- The ML model is trained on synthetic data; real-world accuracy is unknown.
- Non-identity fields (MRP, description) are not fingerprinted.

17.2 Future Work

- Deploy to a public L2 or permissioned network; independent verification in the app via a light client or block explorer links.
- Per-user key custody (HSM/KMS, MPC wallets or account abstraction).
- Unit-level serialisation with secure elements (NFC) or scratch-off codes revealed at sale.
- Retraining the anomaly model on real scan data; per-category thresholds.
- HTTPS deployment, audit-log integrity, iOS client, multi-tenant onboarding.

Appendix A. Demo Data

Accounts (password `Demo@1234`):

Email	Role	Organisation
admin@veritrace.demo	Admin	VeriTrace Admin
maker@veritrace.demo	Manufacturer	Aurora Electronics
maker2@veritrace.demo	Manufacturer	Nila Pharma
supplier1@veritrace.demo	Supplier	Southern Distributors
supplier2@veritrace.demo	Supplier	Metro Wholesale
retailer1@veritrace.demo	Retailer	CityMart Bengaluru
retailer2@veritrace.demo	Retailer	QuickCare Pharmacy

ID	Product	Situation	Expected
VT-2026-000001	Aurora X1 Smartphone	Full journey, sold	Genuine
VT-2026-000002	Aurora SoundPods Pro	At retailer	Genuine
VT-2026-000003	Aurora 65W GaN Charger	At supplier	Flagged
VT-2026-000004	Aurora PowerBank 20000	Transfer pending	Flagged
VT-2026-000005	Nila Paracetamol 500 mg	Record tampered	Suspect
VT-2026-000006	Nila Cough Syrup	Expired	Flagged
VT-2026-000007	Aurora Buds Lite	Over-scanned (clone)	Flagged
VT-2026-000008	Aurora Tab 11	At manufacturer	Flagged
VT-2026-000009 – 13	Vitamin C, Sunscreen, ORS drink, Dash camera, Titanium smartwatch	Sold / at retailer	Genuine
VT-2026-000014	Aurora Smart Bulb	At manufacturer	Flagged
Forged QR	Real ID, fake fingerprint	Counterfeit label	Suspect
VT-2026-999999	Unregistered	No record	Unverified

Appendix B. Glossary

Term	Definition
Smart contract	Program deployed on a blockchain whose state changes only through its functions.
SHA-256	Cryptographic hash function producing a 256-bit fingerprint.
keccak256	Ethereum's hash function, used to derive the on-chain product key.
msg.sender	The account that signed the current transaction.
HD wallet	Hierarchical deterministic wallet: many accounts derived from one seed phrase.
IsolationForest	Unsupervised anomaly-detection algorithm that isolates outliers with random trees.
Bloc	Flutter state-management pattern separating events, state and UI.
adb reverse	Android tool feature forwarding a phone port to the computer over USB.

Appendix C. Technology Stack

Layer	Technologies
Mobile	Flutter 3.44, Dart 3.12, flutter_bloc, equatable, go_router, dio, mobile_scanner, qr_flutter, share_plus, pdf, printing, firebase_core, firebase_auth, shared_preferences
Backend	Node.js 22, Express 4, ethers 6, firebase-admin 13, zod, qrcode, jest, supertest
Blockchain	Solidity 0.8.24, Hardhat 2.29, hardhat-toolbox, chai
Data / identity	Google Cloud Firestore, Firebase Authentication
ML	Python 3.12, FastAPI, uvicorn, scikit-learn, numpy, joblib, pytest
Tooling	PowerShell scripts (setup, start-all, stop-all, package-client), Postman collection